

Prova: Virtual Try-On and AI Shopping Assistant for Fashion E-commerce

YOUR NAME

YOUR UNIVERSITY
YOUR FACULTY OR COLLEGE
YOUR DEPARTMENT

Bachelor Graduation Project
May 2026

Abstract

Write a concise abstract describing the motivation, methods, and outcomes of Prova. Keep it to about 200 to 300 words.

Acknowledgements

Optional. Acknowledge supervisors, colleagues, and family.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Definition	2
1.3 Problem Solution	3
1.4 Key Features	4
1.5 Project Contributions	5
1.6 Scope and Limitations	5
1.6.1 In Scope	5
1.6.2 Out of Scope	6
1.6.3 Known Limitations	6
1.7 Thesis Organization	7
2 Literature Review	8
2.1 Overview	8
2.2 Image-Based Virtual Try-On	8
2.3 Multiview Virtual Try-On	10
2.4 Generative Models for Try-On	10
2.5 360 View Synthesis and 3D Reconstruction	11
2.6 Human Body Understanding and Preprocessing	12
2.7 Body Measurement Estimation	13
2.8 Retrieval-Augmented Discovery for Fashion E-Commerce	13
2.9 Summary	15
3 System Analysis	16
3.1 Overview	16
3.2 Stakeholders	16
3.3 Functional Requirements	17

3.3.1	Authentication and User Management	17
3.3.2	Marketplace and E-Commerce	17
3.3.3	Community and Social Features	18
3.3.4	Virtual Try-On	18
3.3.5	Body Measurement and Size Recommendation	19
3.3.6	360-Degree Visualization	19
3.3.7	AI Fashion Assistant (RAG)	19
3.4	Non-Functional Requirements	20
3.5	Use Case Model	21
3.5.1	UC-1: Browse and Purchase Products	22
3.5.2	UC-2: Virtual Try-On	22
3.5.3	UC-3: Body Measurement and Size Recommendation	23
3.5.4	UC-4: AI Fashion Assistant Chat	23
3.5.5	UC-5: Community Participation	24
3.5.6	UC-6: Merchant Product Management	24
3.6	Data Model and Entity-Relationship Diagram	24
3.6.1	Core Entities	25
3.6.2	Community Entities	26
3.6.3	Key Relationships	27
3.7	Class Diagram	27
3.8	Process and Sequence Diagrams	28
3.8.1	Product Purchase Flow	28
3.8.2	Virtual Try-On Flow	29
3.8.3	RAG Fashion Assistant Flow	30
3.8.4	Body Measurement Flow	30
3.8.5	Multiview Try-On and 360 Reconstruction Flow	31
3.9	Summary	31
4	Proposed System Design	32
4.1	Overview	32
4.2	Architecture Overview	32
4.3	Frontend Design	33
4.3.1	Framework and Routing	33
4.3.2	State Management and API Communication	34
4.3.3	Internationalization (i18n)	34
4.4	Backend Design	34
4.4.1	Module Architecture	34
4.4.2	Middleware Pipeline	35
4.4.3	User Caching	36

4.4.4	Error Handling	36
4.5	Try-On Service Design	36
4.5.1	Service Architecture	36
4.5.2	Multi-Angle Try-On Pipeline Design	37
4.5.3	Multiview (VTON360) Pipeline Design	38
4.5.4	Measurement Pipeline Design	38
4.6	Chat Service (RAG) Design	39
4.6.1	Pipeline Architecture	39
4.6.2	Conversation History	40
4.6.3	Knowledge Base Integration	40
4.6.4	Embedding Strategy	40
4.7	API Design	41
4.7.1	Backend API Conventions	41
4.7.2	Try-On Service API	41
4.7.3	Chat Service API	42
4.8	Security and Privacy Design	42
4.8.1	Authentication Flow	42
4.8.2	Password Security	43
4.8.3	Rate Limiting	43
4.8.4	Input Validation and Sanitization	43
4.8.5	Data Privacy	43
4.9	Deployment Architecture	43
4.10	Summary	44
5	Implementation	45
5.1	Overview	45
5.2	Backend Implementation	45
5.3	Frontend Implementation	45
5.4	Try-On Service Implementation	45
5.5	Body Measurement Implementation	46
5.6	RAG System Implementation	46
5.7	Models and Equations	46
5.8	Datasets	46
5.9	Results	46
6	Testing	47
6.1	Testing Strategy	47
6.2	Unit Testing	47
6.3	Integration Testing	47

6.4	Performance Testing	47
6.5	User Testing	47
6.6	Summary	48
7	Conclusion and Future Work	49
7.1	Conclusion	49
7.2	Future Work	49

List of Figures

3.1	Use case diagram for Prova.	21
3.2	Entity-relationship diagram for Prova.	25
3.3	Class diagram showing major components of Prova.	27
3.4	Sequence diagram for the product purchase flow.	28
3.5	Sequence diagram for the 2D virtual try-on flow.	29
3.6	Sequence diagram for the RAG fashion assistant pipeline.	30
3.7	Sequence diagram for body measurement and size recommendation. . . .	30
4.1	High-level service-oriented architecture of Prova.	33

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

The global fashion e-commerce market has grown rapidly over the past decade, driven by increasing internet penetration, mobile commerce adoption, and shifting consumer preferences toward online shopping. However, this growth has been accompanied by persistently high product return rates, with clothing consistently ranking among the most returned categories in online retail. A significant portion of these returns stem from fit and appearance dissatisfaction—consumers receive garments that look different from what they expected based on static product photographs and generic size charts.

Traditional product presentation in fashion e-commerce relies on a small set of studio photographs showing a garment on a single model or laid flat on a surface. These images provide limited information about how the garment would look on a specific consumer’s body shape, skin tone, and proportions. Size charts, typically presented as static tables of numeric measurements, require consumers to measure themselves accurately and then mentally map those measurements to an unfamiliar sizing system. This process is error-prone even for experienced shoppers, and the problem is compounded by inconsistent sizing across brands and regions.

At the same time, recent advances in computer vision and generative modeling have made several capabilities practically feasible that were previously confined to research prototypes. Diffusion-based image synthesis can now produce photorealistic garment transfers. 3D Gaussian splatting enables real-time novel view rendering from sparse image inputs. Body shape regression models can estimate anthropometric measurements from a single photograph. Dense retrieval systems powered by sentence embeddings can understand natural language shopping queries and match them against product catalogs. These technologies have individually reached sufficient maturity for integration into consumer-facing applications, but they are typically developed and deployed as isolated capabilities rather than components of a unified shopping experience.

This gap between the maturity of individual AI components and the absence of integrated platforms that combine them into a coherent user experience motivates Prova. The project aims to demonstrate that virtual try-on, body measurement, 360-degree visualization, and intelligent product discovery can be integrated into a single fashion e-commerce platform without requiring novel fundamental research in any individual area, but rather through careful systems design, service orchestration, and engineering.

1.2 Problem Definition

Online clothing purchases suffer from several interrelated challenges that collectively degrade the shopping experience and drive return rates:

Uncertainty about fit and appearance. Consumers cannot physically try on garments before purchasing. Static product images show how a garment looks on a specific model under controlled studio conditions, but provide little insight into how the same garment would look on the consumer’s own body. This uncertainty is particularly acute for garments where fit is critical, such as tailored shirts, dresses, and formal wear. The lack of personalized visualization forces consumers to make purchasing decisions based on incomplete information.

Difficulty choosing sizes. Size charts vary significantly across brands, regions, and garment categories. A consumer who wears size M in one brand may need size L in another. Converting between regional sizing systems (EU, US, UK) adds further confusion. Without reliable measurement-based guidance, consumers frequently order multiple sizes of the same garment with the intention of returning those that do not fit, increasing logistical costs and environmental waste.

Limited personalization and discovery. Large product catalogs contain hundreds or thousands of items, making it difficult for consumers to find garments that match their preferences, body type, occasion, and budget. Traditional keyword-based search requires consumers to know exact product terminology, and basic category filters provide only coarse-grained navigation. Consumers seeking outfit recommendations or style guidance for specific occasions receive no intelligent assistance from conventional e-commerce interfaces.

Fragmented user experiences. Fashion shopping involves multiple activities beyond product search and purchase: seeking style inspiration, discussing trends with peers, exploring designer collections, and receiving personalized recommendations. In most existing platforms, these activities are distributed across separate applications and websites, creating a fragmented experience that reduces engagement and makes it difficult for consumers to move seamlessly between discovery, visualization, and purchase.

Absence of intelligent assistance. Existing chatbot and search interfaces in fashion e-commerce typically rely on keyword matching or rigid decision trees. They cannot

understand nuanced natural language queries such as “a casual summer outfit for a beach wedding under 200 dollars” or provide contextual recommendations that account for the consumer’s body measurements, style preferences, and available inventory.

Any proposed solution must address these challenges while operating under practical constraints: variable input image quality from consumer devices, acceptable response times for interactive use, support for diverse body types and garment categories, and a modular architecture that allows individual components to evolve independently as underlying models improve.

1.3 Problem Solution

Prova addresses these challenges through a modular platform architecture that combines a web frontend, a backend API, and specialized AI services into a cohesive shopping experience. Rather than building a monolithic application, the system is decomposed into independently deployable services that communicate through well-defined APIs, enabling each component to be developed, tested, and scaled independently.

The **frontend** delivers a bilingual (Arabic and English) web-based e-commerce interface built with Next.js. It provides the full shopping workflow: product browsing with filtering and search, detailed product pages, shopping cart and checkout, order management, and wishlists. Beyond core e-commerce, the frontend includes community spaces where users can share posts and engage with other shoppers, designer pages that showcase curated collections, and an interactive virtual try-on interface where users upload their photographs and visualize garments on their own body.

The **backend** serves as the central orchestration layer, managing authentication and authorization, product and merchant management, order processing, community content moderation, and coordination with AI services. It exposes RESTful APIs consumed by the frontend and handles the routing of AI requests to the appropriate specialized service.

The **try-on service** provides multiple AI capabilities through a FastAPI application. The 2D virtual try-on pipeline uses OOTDiffusion, a diffusion-based model that fuses garment appearance into person images without explicit geometric warping. A multi-view extension generates try-on results from multiple viewpoints, enabling consumers to see how a garment looks from the front, sides, and back. The preprocessing pipeline uses DensePose for dense body surface estimation and human parsing for semantic segmentation of clothing regions, providing the conditioning inputs required by the try-on model. A body measurement module uses SHAPY, a 3D body mesh regression model, to estimate anthropometric measurements from a single front-facing photograph. These measurements are mapped to size recommendations using regional size charts and anthropometric ratios. A 360-degree reconstruction module uses Gaussian splatting (Splatfacto) to generate interactive 360-degree previews from multiview try-on outputs.

The **retrieval and recommendation service** provides intelligent product discovery through a retrieval-augmented generation (RAG) pipeline. The system classifies user intent, extracts structured query attributes (garment type, color, budget, occasion, size) from natural language messages, retrieves candidate products through hybrid search combining MongoDB metadata filters with Sentence-BERT semantic ranking, assembles complete outfit recommendations, and generates bilingual natural language responses grounded in actual catalog data. A knowledge base provides profile-specific styling guidance based on the user’s body measurements and preferences.

This separation of concerns ensures that improvements to any individual AI model—a better try-on model, a more accurate measurement estimator, a faster 3D reconstruction method—can be integrated without disrupting the rest of the platform.

1.4 Key Features

- **Integrated marketplace.** A full e-commerce platform with merchant storefronts, product listings with multi-image galleries, size and color selection, shopping cart, checkout, order tracking, reviews, and wishlists.
- **Personalized item discovery.** A conversational AI assistant that understands natural language queries in Arabic and English, retrieves relevant products from the catalog using hybrid semantic and metadata search, and assembles complete outfit recommendations with explanations.
- **Virtual try-on.** Diffusion-based garment transfer that generates realistic images of the user wearing selected garments. Supports single-view and multi-angle outputs to show the garment from multiple perspectives.
- **Body measurement and size recommendation.** Automated measurement estimation from a single front-facing photograph using 3D body mesh regression, with calibration against user-provided height and weight. Measurements are mapped to regional size recommendations (EU, US, UK) with configurable fit preferences.
- **360-degree visual previews.** Interactive 360-degree views generated from multiview try-on outputs using Gaussian splatting, enabling users to freely rotate the visualized result.
- **Community and designer spaces.** Social features including user posts, design showcases, comments, votes, and follow relationships, creating engagement beyond transactional shopping.
- **Bilingual support.** Full Arabic and English localization across the frontend interface and the AI assistant, with locale-based routing and right-to-left layout support.

1.5 Project Contributions

This project makes the following contributions:

- **An integrated AI-driven fashion platform.** The primary contribution is the integration of multiple AI capabilities—virtual try-on, body measurement, 360 visualization, and retrieval-augmented discovery—into a single, functional e-commerce platform. This demonstrates the feasibility of unifying research-grade AI components into a production-oriented application.
- **A modular service architecture for AI orchestration.** The system demonstrates a practical architecture for coordinating heterogeneous AI services (diffusion models, 3D reconstruction, embedding-based retrieval, body mesh regression) through a unified API layer, with health monitoring, error handling, and analytics across all services.
- **A preprocessing and conditioning pipeline.** The project implements a complete preprocessing workflow that transforms raw user inputs (photographs, videos) into the structured conditioning data (DensePose maps, parsing masks, camera intrinsics) required by downstream AI models.
- **A hybrid retrieval pipeline for fashion discovery.** The RAG system combines structured metadata filtering with dense semantic ranking, intent classification, outfit assembly, and profile-aware knowledge base guidance, creating a retrieval pipeline specifically designed for fashion product catalogs.
- **A measurement-to-size recommendation pipeline.** The system chains 3D body mesh regression with anthropometric ratio estimation and multi-region size chart lookup, providing end-to-end measurement and sizing from a single photograph.

1.6 Scope and Limitations

1.6.1 In Scope

This project encompasses the following:

- A complete web-based e-commerce platform with marketplace, community, and designer features.
- A 2D virtual try-on pipeline using diffusion-based synthesis (OOTDiffusion) with single-view and multi-angle support.
- A preprocessing pipeline using DensePose and human parsing for body surface estimation.

- Body measurement estimation using SHAPY 3D mesh regression with size chart mapping.
- A 360-degree reconstruction pipeline using Gaussian splatting (Splatfacto).
- A retrieval-augmented discovery system with hybrid search, intent classification, and bilingual support.
- A multiview virtual try-on workflow inspired by the VTON360 research pipeline.

1.6.2 Out of Scope

The following areas are explicitly excluded:

- Physics-based cloth simulation and draping.
- Training or fine-tuning of foundational generative models from scratch.
- Payment gateway integration and real financial transactions.
- Mobile native applications (the platform is web-based only).
- Large-scale production deployment with load balancing and CDN infrastructure.

1.6.3 Known Limitations

The current implementation has several known limitations:

- Real-time performance is achieved only for the 360 rendering component; other AI pipelines (try-on, measurement, RAG) operate at interactive but not real-time speeds, with inference times ranging from seconds to tens of seconds depending on the model and hardware.
- Try-on quality depends on input image quality, pose, and lighting conditions. Results degrade for heavily occluded poses, low-resolution inputs, or unusual camera angles.
- Body measurement accuracy is limited by the inherent ambiguity of monocular depth estimation and is affected by clothing, pose, and camera perspective.
- The system relies on external model checkpoints and pretrained weights; it does not include novel model architectures or training procedures.

1.7 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2: Literature Review reviews related work in image-based virtual try-on, multiview try-on, generative models, 3D reconstruction, human body understanding, body measurement estimation, and retrieval-augmented discovery for fashion e-commerce. The chapter traces the evolution of methods in each area and identifies the gaps that motivate an integrated platform.

Chapter 3: System Analysis presents the system requirements, stakeholder analysis, and modeling artifacts including use case diagrams, entity-relationship diagrams, class diagrams, and sequence diagrams that define the scope and structure of the system.

Chapter 4: Proposed System Design describes the system architecture, component design, data flow, algorithmic stages, and the models and technologies selected for each subsystem.

Chapter 5: Implementation details the implementation of each service, including backend API structure, frontend interface, try-on pipeline, measurement module, 360 reconstruction, and RAG system. It also presents the datasets used for evaluation and the quantitative and qualitative results.

Chapter 6: Testing describes the testing methodology, including unit tests, integration tests, performance measurements, and user evaluation sessions.

Chapter 7: Conclusion and Future Work summarizes the project outcomes, discusses lessons learned, and outlines directions for future improvement.

Chapter 2

Literature Review

2.1 Overview

This chapter reviews the research areas that inform Prova: image-based virtual try-on, multiview try-on and view synthesis, generative models for conditioned image synthesis, 3D reconstruction, human body understanding and measurement estimation, and retrieval-augmented discovery for fashion e-commerce. Rather than surveying each topic in isolation, the discussion traces how methods in each area have evolved, identifies their respective limitations, and highlights how their integration motivates a unified fashion platform. The progression from single-view garment transfer toward multiview-consistent, measurement-aware, and retrieval-informed shopping experiences forms the central thread of this review.

2.2 Image-Based Virtual Try-On

Image-based virtual try-on aims to synthesize a realistic image of a person wearing a target garment, given only a person photograph and a garment image as input. Early approaches decompose this problem into two stages: geometric alignment of the garment to the person’s body, followed by pixel-level refinement to blend the warped garment into the scene.

VITON introduced the foundational two-stage pipeline, combining a coarse result from a clothing-agnostic representation with a refinement network that sharpens textures and corrects boundary artifacts [1]. While VITON demonstrated the viability of image-conditioned garment transfer, it suffered from noticeable misalignment under non-trivial poses and produced blurred textures in complex garment regions. CP-VTON improved upon this by introducing a geometric matching module based on thin-plate spline (TPS) transformation, which better aligns the garment to the target pose before synthesis [2]. The explicit geometric warping step reduced spatial misalignment, but

the approach remained sensitive to large pose differences and struggled with fine-grained texture preservation.

Subsequent methods shifted toward higher-resolution outputs and more robust warping strategies. HR-VITON addressed the resolution bottleneck by incorporating a misalignment-aware segmentation module and a conditional generator that operates at higher spatial dimensions [3]. This approach significantly improved visual quality at 1024×768 resolution, though it introduced higher computational cost and required carefully curated training data at the target resolution.

The emergence of diffusion-based generative models marked a further shift in try-on quality. StableVITON adapted latent diffusion models for virtual try-on by introducing zero-cross-attention blocks that inject garment features into the denoising process without requiring explicit warping [4]. This eliminated the need for separate geometric transformation modules and improved texture fidelity, particularly for patterned and textured garments. However, the approach inherited the computational cost of diffusion sampling, making real-time deployment challenging.

IDM-VTON extended the diffusion-based paradigm by combining high-level garment semantics with low-level texture features through a dual-encoder architecture [5]. The model uses a TrueNet component for identity preservation and achieves state-of-the-art results on standard benchmarks. Nevertheless, like other diffusion-based methods, inference latency remains a practical concern for interactive applications.

OOTDiffusion proposed an alternative approach by formulating try-on as outfitting fusion within a pretrained latent diffusion framework [6]. Rather than conditioning the diffusion process on warped garment features, OOTDiffusion fuses garment appearance through outfitting dropout and attention-based feature injection. This design preserves garment details more faithfully than warp-based methods and supports both half-body and full-body try-on scenarios. However, OOTDiffusion operates in single-view mode and does not address multiview consistency, limiting its applicability for 360-degree visualization.

Across these methods, a clear progression emerges: from explicit geometric warping (VITON, CP-VTON) to learned alignment at higher resolutions (HR-VITON), and finally to diffusion-based synthesis that bypasses warping entirely (StableVITON, IDM-VTON, OOTDiffusion). Each generation improves texture fidelity and pose robustness, but introduces new tradeoffs in computational cost, training data requirements, and deployment complexity. The challenge of producing consistent results across multiple viewpoints remains largely unaddressed by single-view methods, motivating the multiview extensions discussed in the following section.

2.3 Multiview Virtual Try-On

Single-view try-on outputs, while increasingly realistic, provide only a partial visualization of how a garment looks on a person. In real shopping scenarios, consumers benefit from seeing the garment from multiple angles, which requires multiview consistency—the garment must appear coherent in color, texture, and fit across different viewpoints.

VTON 360 formulates 3D try-on as a multiview extension of 2D pipelines [7]. The method introduces view-consistent conditioning to preserve garment identity across viewpoints, using multiview priors and cross-view attention mechanisms to enforce coherence between independently generated views. This approach demonstrates that multiview conditioning and geometric priors can significantly improve consistency compared to naive per-view synthesis, where each viewpoint is generated independently and often exhibits inconsistent garment appearance, color shifts, or structural discontinuities.

However, multiview try-on methods face several challenges that remain active areas of research. First, maintaining fine-grained texture consistency across large angular differences (for example, between front and back views) is difficult because different viewpoints expose different garment regions with minimal overlap. Second, occlusion handling becomes more complex in side and back views, where the garment may be partially hidden by the body or other clothing. Third, the computational cost scales with the number of views, as each view typically requires a separate forward pass through the generation model. These challenges motivate the use of explicit 3D representations for view synthesis, discussed in Section 2.5, as a complementary approach to multiview-conditioned 2D generation.

2.4 Generative Models for Try-On

The quality improvements observed in recent try-on systems are closely tied to advances in conditional generative modeling, particularly diffusion models. Understanding these foundations is essential for contextualizing the architectural choices in modern try-on pipelines.

Denoising diffusion probabilistic models (DDPMs) introduced a principled framework for iterative image generation through a forward noising process and a learned reverse denoising process [8]. DDPMs produce high-fidelity samples and offer stable training dynamics compared to earlier generative adversarial networks (GANs), but their iterative sampling requires hundreds of denoising steps, resulting in slow inference.

Latent diffusion models (LDMs) addressed this computational bottleneck by performing the diffusion process in a compressed latent space rather than directly in pixel space [9]. By encoding images into a lower-dimensional latent representation using a pretrained autoencoder, LDMs reduce memory and computation requirements while preserving per-

ceptual quality. The introduction of cross-attention conditioning in LDMs enables flexible integration of external signals such as text prompts, segmentation masks, or reference images, making LDMs particularly well-suited for conditioned synthesis tasks including virtual try-on.

These advances have directly influenced the try-on literature. StableVITON, IDM-VTON, and OOTDiffusion all build upon the LDM framework, leveraging its conditioning mechanisms to inject garment information into the generation process. The key design question across these systems is how to encode garment appearance—whether through warped feature maps, cross-attention injection, or outfitting fusion—such that the generated output faithfully reproduces garment details while respecting the person’s pose and body shape. This interplay between generative model architecture and garment conditioning strategy remains a central research challenge, with each approach offering different tradeoffs between fidelity, consistency, and inference speed.

2.5 360 View Synthesis and 3D Reconstruction

While multiview try-on generates a discrete set of viewpoint images, full 360-degree visualization requires continuous novel view synthesis or explicit 3D reconstruction. This capability enables interactive product previews where users can freely rotate the visualized result, providing a richer understanding of garment fit and appearance.

Neural Radiance Fields (NeRF) represent a scene as a continuous volumetric function that maps 3D coordinates and viewing directions to color and density values [10]. By training on a sparse set of posed images, NeRF can synthesize photorealistic novel views through volumetric rendering. NeRF demonstrated remarkable visual quality for view synthesis, but its reliance on per-scene optimization and expensive volumetric ray marching makes it impractical for real-time applications. Training a single NeRF scene can take hours, and rendering a single frame requires evaluating the network at hundreds of sample points per ray.

3D Gaussian Splatting (3DGS) addressed these limitations by representing scenes as collections of 3D Gaussian primitives that are rasterized using a differentiable splatting operation [11]. This explicit point-based representation enables real-time rendering at high resolution while maintaining visual quality comparable to NeRF. Gaussian splatting also supports faster optimization, with training times measured in minutes rather than hours for typical scenes. The Nerfstudio framework provides modular implementations of both NeRF and Gaussian splatting variants, including Splatfacto, which offers a production-ready Gaussian splatting pipeline with configurable densification, pruning, and appearance modeling strategies [12].

For fashion applications, the choice between NeRF-based and splatting-based methods involves clear tradeoffs. NeRF provides smoother interpolation in regions with sparse

observations but cannot achieve interactive frame rates. Gaussian splatting achieves real-time rendering and faster training but may produce artifacts in under-observed regions where Gaussian primitives are insufficiently constrained. For an interactive e-commerce platform where users expect responsive 360-degree previews, the real-time rendering capability of Gaussian splatting is a decisive advantage, even at the cost of occasional visual artifacts in challenging viewpoints.

2.6 Human Body Understanding and Preprocessing

Virtual try-on and body measurement pipelines require accurate understanding of human body structure as a preprocessing step. This involves detecting and segmenting the person in the input image, estimating body pose, and extracting dense correspondences between the image and a canonical body surface. The quality of these preprocessing outputs directly affects the accuracy of downstream tasks.

DensePose extends the problem of human pose estimation from sparse keypoints to dense surface correspondences [13]. Given an input image, DensePose predicts a mapping from every pixel belonging to a person to a UV coordinate on the SMPL body surface [14]. This dense correspondence map provides the geometric foundation for garment warping in try-on pipelines, as it enables precise spatial alignment between the input image and a canonical body representation. DensePose predictions are typically produced using architectures built on top of the Detectron2 framework, which provides modular object detection and segmentation components [15].

Person detection, which precedes DensePose and parsing, is commonly handled by general-purpose object detectors. The YOLO family of detectors, including YOLOv8, provides fast and accurate bounding box detection suitable for identifying persons in unconstrained images [16], [17]. In preprocessing pipelines, person detection serves as the initial step that crops and isolates the relevant region before applying more expensive per-pixel analysis such as DensePose and human parsing.

Human parsing complements DensePose by providing semantic segmentation of clothing regions, labeling each pixel as belonging to categories such as upper body, lower body, dress, hair, or skin. Together, DensePose correspondences and parsing maps provide the conditioning inputs that modern try-on models require to accurately place garments on the person’s body.

The accuracy and robustness of these preprocessing components are critical. Errors in person detection lead to incorrect crops, DensePose failures produce garbled surface mappings, and parsing mistakes cause garments to bleed into skin or hair regions. In practice, building a reliable preprocessing pipeline requires careful orchestration of multiple models and handling of edge cases such as occluded bodies, unusual poses, and low-resolution inputs.

2.7 Body Measurement Estimation

Accurate size recommendation in fashion e-commerce requires estimating body measurements from visual input. Traditional approaches rely on manual measurement or depth sensors, but image-based methods offer a more accessible alternative that requires only a standard photograph.

Parametric human body models provide the geometric foundation for measurement estimation. SMPL represents the human body as a differentiable function of shape and pose parameters, producing a 3D mesh from a compact parameter vector [14]. Shape parameters control body proportions such as height, weight distribution, and limb lengths, while pose parameters control joint rotations. By fitting SMPL parameters to visual observations, systems can recover a 3D body mesh from which anthropometric measurements can be directly computed.

SHAPY advanced this approach by training a regression network that predicts SMPL body shape parameters directly from a single image, using linguistic shape attributes as additional supervision [18]. Unlike earlier fitting-based methods that require iterative optimization, SHAPY produces shape estimates in a single forward pass, enabling faster inference. The model was trained on a dataset annotated with natural language descriptions of body shape, which provides a richer supervisory signal than mesh-only losses. SHAPY demonstrates that accurate body shape estimation—and by extension, measurement extraction—is feasible from monocular images without requiring depth sensors or multiple viewpoints.

However, image-based measurement estimation faces inherent limitations. Clothing, pose variation, and camera perspective introduce systematic biases that affect circumference and width estimates. Loose clothing obscures the true body contour, leading to overestimation of measurements. Camera distance and lens distortion affect apparent proportions, requiring calibration with a known reference measurement such as the person’s height. These challenges motivate hybrid approaches that combine model-predicted shape with user-provided reference values to correct for systematic errors, an approach adopted in the system described in this thesis.

2.8 Retrieval-Augmented Discovery for Fashion E-Commerce

Product discovery in fashion e-commerce requires systems that understand natural language queries, match them against large product catalogs, and return relevant results with explanations. Traditional keyword-based search fails to capture the semantic richness of fashion queries, which often involve abstract concepts such as style, occasion, and

aesthetic preference rather than exact product names.

Retrieval-augmented generation (RAG) provides a framework for combining retrieval and generation in a single pipeline [19]. In the RAG paradigm, a retriever first identifies relevant documents from an external knowledge base, and a generator then produces a response conditioned on both the user query and the retrieved context. This architecture grounds generated responses in factual catalog data, reducing hallucination and ensuring that recommendations correspond to real, available products.

The retrieval component in fashion applications typically relies on dense embedding models. Sentence-BERT produces fixed-dimensional sentence embeddings through a Siamese network architecture, enabling efficient similarity computation via cosine distance [20]. When applied to product catalogs, each product is embedded based on its textual description (name, material, color, style tags), and user queries are embedded in the same space. Retrieval then becomes a nearest-neighbor search in embedding space, which scales to large catalogs using vector databases or approximate nearest-neighbor indices.

Fashion-specific retrieval introduces additional challenges beyond general semantic search. Product descriptions are often short and formulaic, requiring the embedding model to capture subtle distinctions between similar items. Color matching requires fuzzy expansion, as users may request “blue” while the catalog contains entries labeled “navy,” “cobalt,” or “sky blue.” Size and availability constraints introduce hard filters that must be applied before or alongside semantic ranking. Budget and occasion requirements add further dimensions that pure embedding similarity cannot capture.

These considerations motivate hybrid retrieval architectures that combine structured metadata filtering with semantic ranking. A two-stage pipeline—first applying hard filters on attributes such as stock, gender, price range, and size availability, then ranking the filtered candidates by semantic similarity—balances precision and recall while remaining computationally efficient. Progressive filter relaxation, where constraints are loosened if too few candidates survive the initial filter, ensures that the system degrades gracefully rather than returning empty results.

Intent classification adds another layer of intelligence to the retrieval pipeline. Not every user message is a product search; greetings, follow-up questions, and non-fashion queries require different handling. Classifying user intent before invoking the retrieval pipeline avoids unnecessary computation and enables the system to respond appropriately to conversational inputs. This classification step, combined with structured intent extraction that identifies specific attributes such as garment type, color, and occasion from the user’s message, transforms an unstructured natural language query into a structured retrieval request.

The integration of retrieval, intent understanding, and response generation into a coherent pipeline connects the RAG literature to the broader goal of intelligent shop-

ping assistance. Rather than treating product search as an isolated information retrieval problem, the system must orchestrate multiple components—classification, extraction, retrieval, ranking, and generation—into a responsive conversational experience. This systems integration perspective, rather than any single algorithmic contribution, distinguishes applied fashion RAG systems from the foundational NLP research on which they build.

2.9 Summary

The literature reviewed in this chapter spans several distinct but interconnected research areas. Image-based virtual try-on has progressed from explicit geometric warping to diffusion-based synthesis, steadily improving texture fidelity and pose robustness while introducing new tradeoffs in computational cost. Multiview extensions address the consistency gap left by single-view methods but remain computationally expensive and sensitive to large viewpoint changes. 3D reconstruction techniques, particularly Gaussian splatting, enable real-time 360-degree visualization that complements multiview try-on outputs. Human body understanding through dense pose estimation and body measurement regression provides the geometric and anthropometric foundations that try-on and sizing systems require. Finally, retrieval-augmented discovery combines semantic search with structured filtering and conversational intent understanding to support intelligent product exploration.

No single existing system integrates all of these capabilities into a unified platform. Individual research contributions advance specific components—better try-on quality, faster 3D rendering, more accurate measurements, smarter retrieval—but they are typically evaluated in isolation. The contribution of Prova lies in combining these components into an integrated architecture that supports the full shopping workflow: discovery, visualization, measurement, and purchase. The design decisions and implementation details of this integration are presented in the subsequent chapters.

Chapter 3

System Analysis

3.1 Overview

This chapter presents the requirements analysis and modeling artifacts for Prova. It identifies the system stakeholders, defines the functional and non-functional requirements, and documents the system structure through use case descriptions, an entity-relationship diagram, a class diagram, and process diagrams. These artifacts establish the scope and constraints that guide the design and implementation described in subsequent chapters.

3.2 Stakeholders

The system serves five primary stakeholder groups, each with distinct goals and interaction patterns:

- **Shoppers (End Users).** Consumers who browse products, use virtual try-on and body measurement features, interact with the AI fashion assistant, participate in the community, and complete purchases. They expect personalized recommendations, accurate size guidance, realistic garment visualization, and a seamless bilingual interface.
- **Merchants.** Business owners who register merchant accounts, list products with images and size charts, manage inventory and pricing, fulfill orders, and monitor sales through a merchant dashboard. They require tools for product management, order processing, and customer communication.
- **Customer Service Representatives.** Support agents assigned to branches who handle user inquiries through a ticketing system. They receive, respond to, and resolve support tickets with prioritized workflows (open, assigned, resolved, closed). Each agent has a customer service profile linked to a specific branch, and they communicate with users through threaded ticket messages with unread tracking.

- **Designers.** Users with the designer role who submit original design concepts for community review. Designs go through an approval workflow (pending, approved, in production, published, rejected) managed by administrators. Designers benefit from community feedback through upvotes, downvotes, and comments.
- **System Administrators.** Platform operators who manage user accounts, moderate community content, approve or reject designer submissions, oversee support operations, and monitor system health across all services.

3.3 Functional Requirements

The functional requirements are organized by subsystem to reflect the modular architecture.

3.3.1 Authentication and User Management

- FR-1. The system shall support user registration with email, password, phone, and OTP verification.
- FR-2. The system shall support login via local credentials and Google OAuth.
- FR-3. The system shall issue JWT access tokens and refresh tokens for session management.
- FR-4. The system shall support password reset via email OTP.
- FR-5. The system shall enforce role-based access control with five roles: user, merchant, admin, customer service, and designer.
- FR-6. The system shall allow users to manage their profile, including profile image, body measurements (height, weight), and a dedicated try-on image.

3.3.2 Marketplace and E-Commerce

- FR-7. Merchants shall be able to create, update, and delete product listings with name, description, price, sale price, images, sizes, colors, material, brand, tags, styles, occasions, and size measurement charts.
- FR-8. Products shall support gender categorization (male, female, unisex) and garment type classification.
- FR-9. Shoppers shall be able to browse products with filtering by category, type, brand, price range, color, and size.

- FR-10. Shoppers shall be able to add products to a shopping cart with size and color selection.
- FR-11. The system shall support order creation with delivery address and payment method, order status tracking, and order history.
- FR-12. Shoppers shall be able to submit product reviews with ratings (0.5–5 stars), text comments, likes, and threaded replies.
- FR-13. Shoppers shall be able to add products, posts, and designs to a wishlist with optional notes.
- FR-14. Merchants shall have a dashboard for managing products, viewing orders, and tracking sales.

3.3.3 Community and Social Features

- FR-15. Users shall be able to create community posts of types: issue, review, tip, or question, with title, content, and optional images.
- FR-16. Designers shall be able to submit design proposals with name, description, category, and price.
- FR-17. Users shall be able to comment on posts and designs, with support for threaded replies (parent–child comments).
- FR-18. Users shall be able to vote on posts (like), designs (upvote/downvote), and comments (like).
- FR-19. Users shall be able to follow and unfollow other users, with follower and following counts maintained on user profiles.
- FR-20. The system shall generate notifications for social interactions: likes, comments, replies, votes, design status changes, and new followers.

3.3.4 Virtual Try-On

- FR-21. The system shall accept a person image and a garment image and generate a 2D virtual try-on result using diffusion-based synthesis.
- FR-22. The system shall support multi-angle try-on, generating results from front, right, left, and back viewpoints.
- FR-23. The system shall preprocess person inputs (video or images) through a DensePose and human parsing pipeline to produce conditioning data.

- FR-24. The system shall accept garment front and back images and prepare them for the multiview pipeline.
- FR-25. The system shall support a full multiview try-on workflow: preprocessing, dataset construction, model inference, and result retrieval.
- FR-26. The system shall persist try-on results with metadata (model type, product reference, garment category) and support history retrieval per user.

3.3.5 Body Measurement and Size Recommendation

- FR-27. The system shall estimate body measurements (chest, waist, hip, shoulder, neck, arm length, trouser length, thigh) from a single front-facing photograph.
- FR-28. The system shall accept user-provided height and weight for calibration and apply weight-based circumference correction when the predicted mass deviates significantly.
- FR-29. The system shall map estimated measurements to size recommendations across multiple regional size charts (EU, US, UK, IT, FR, DE) with configurable fit preferences (regular, slim, oversized).
- FR-30. The system shall persist measurement results per user with confidence scores and processing metadata.

3.3.6 360-Degree Visualization

- FR-31. The system shall generate 360-degree interactive previews from multiview try-on outputs using Gaussian splatting reconstruction.
- FR-32. The system shall expose status and health endpoints for the reconstruction service.

3.3.7 AI Fashion Assistant (RAG)

- FR-33. The system shall classify user messages into categories: product search, greeting, follow-up, non-fashion, or clarification.
- FR-34. The system shall extract structured intent attributes (garment type, colors, budget, sizes, brands, occasion, gender) from natural language messages.
- FR-35. The system shall retrieve candidate products using hybrid search: metadata filtering followed by semantic ranking with a configurable similarity threshold.
- FR-36. The system shall assemble retrieved products into complete outfit recommendations.

- FR-37. The system shall generate bilingual (Arabic and English) natural language responses grounded in retrieved catalog data.
- FR-38. The system shall maintain per-user conversation history for multi-turn interactions.
- FR-39. The system shall provide profile-aware styling guidance using a knowledge base that accounts for height, weight, age, and gender.

3.4 Non-Functional Requirements

Category	Requirement
Reliability	The system shall implement health check endpoints for all AI services. Failed service dependencies shall be reported at startup without preventing other services from operating. Global exception handlers shall catch and log unhandled errors with structured error responses.
Performance	Interactive endpoints (product browsing, cart, orders) shall respond within 500ms. AI inference endpoints (try-on, measurements) shall complete within 60 seconds. The RAG pipeline shall respond within 10 seconds for typical queries. The 360 rendering component shall support real-time frame rates.
Security	All sensitive endpoints shall require JWT authentication. Passwords shall be hashed using bcrypt. Rate limiting shall be applied to authentication and API endpoints. CORS shall restrict origins to authorized frontends. Input validation shall be enforced on all endpoints using schema-based validation (Zod). The system shall enforce HTTPS in production and apply security headers.
Scalability	Services shall be independently deployable and horizontally scalable. The backend shall support concurrent connections through connection pooling and async I/O. MongoDB indexes shall be defined for all frequent query patterns.

Category	Requirement
Maintainability	The codebase shall follow a modular architecture with clear separation between routes, controllers, services, and models. Each backend module shall follow a consistent file structure. Configuration shall be externalized through environment variables.
Usability	The frontend shall support Arabic and English with locale-based routing and RTL layout. The interface shall be responsive across desktop and mobile viewports. The AI assistant shall detect input language and respond in the same language.

3.5 Use Case Model

This section describes the primary use cases that define user interactions with Prova. Figure 3.1 presents the use case diagram.

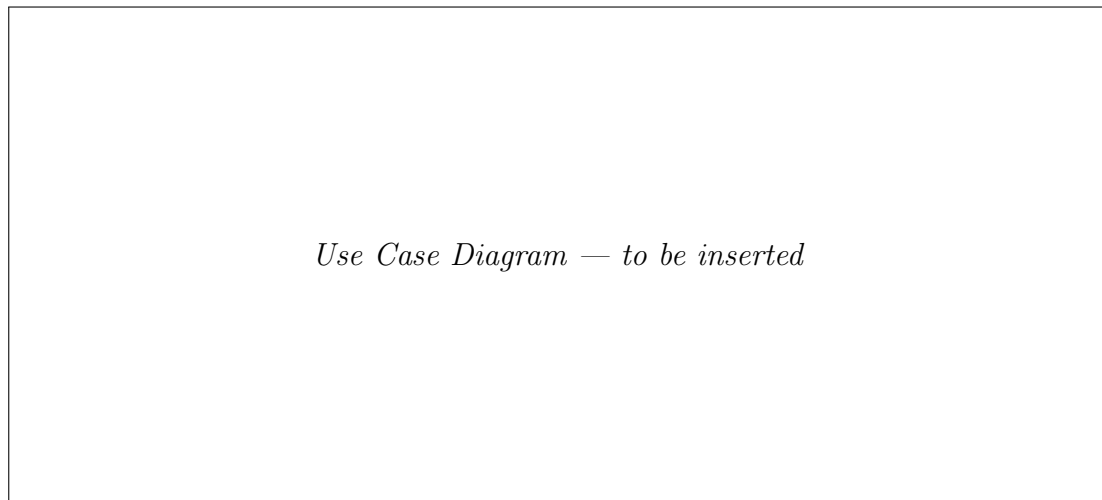


Figure 3.1: Use case diagram for Prova.

3.5.1 UC-1: Browse and Purchase Products

Actor	Shopper
Precondition	User is registered and logged in.
Main Flow	1. User browses the shop with optional filters (category, price, color, size, brand). 2. User selects a product and views details, images, reviews, and size chart. 3. User selects size and color, adds product to cart. 4. User proceeds to checkout, enters delivery address and payment method. 5. System creates the order and decrements product stock. 6. User receives order confirmation.
Postcondition	Order is created with status “pending.” Stock is updated.
Alternate Flow	If product is out of stock, system displays an error and prevents adding to cart.

3.5.2 UC-2: Virtual Try-On

Actor	Shopper
Precondition	User has uploaded a profile try-on image or provides one during the session.
Main Flow	1. User navigates to the virtual try-on page. 2. User selects or uploads a person image and a garment image. 3. System sends images to the try-on service. 4. Try-on service preprocesses inputs and runs diffusion-based synthesis. 5. System returns the generated try-on result image. 6. Result is saved to the user’s try-on history.
Postcondition	Try-on result is displayed and persisted in history.
Alternate Flow	If the try-on service is unavailable, system displays a service status message.

3.5.3 UC-3: Body Measurement and Size Recommendation

Actor	Shopper
Precondition	User has a front-facing full-body photograph.
Main Flow	1. User uploads a front-facing photograph. 2. User enters their height (required) and weight (optional). 3. User selects gender, region (EU/US/UK), and fit preference. 4. System sends the image and parameters to the SHAPY measurement service. 5. Service estimates body measurements from the 3D mesh regression output. 6. System maps measurements to size recommendations and returns results with confidence scores.
Postcondition	Measurements and size recommendation are displayed and saved to user profile.
Alternate Flow	If SHAPY service is unavailable, system informs the user and suggests retrying later.

3.5.4 UC-4: AI Fashion Assistant Chat

Actor	Shopper
Precondition	None (assistant is accessible without login for basic queries).
Main Flow	1. User opens the fashion assistant chat interface. 2. User sends a natural language message (Arabic or English). 3. System classifies the message intent. 4. For product searches: system extracts structured attributes, retrieves matching products via hybrid search, assembles outfit recommendations, and generates a natural language response. 5. Response is displayed with product cards and explanations.
Postcondition	Conversation turn is persisted for multi-turn context.
Alternate Flow	For greetings, non-fashion, or ambiguous queries, the system responds with appropriate fixed messages or clarification questions.

3.5.5 UC-5: Community Participation

Actor	Shopper, Designer
Precondition	User is registered and logged in.
Main Flow	1. User creates a post (issue, review, tip, or question) with title, content, and optional images. 2. Other users view, like, and comment on the post. 3. Post author and commenters receive notifications for interactions. 4. Users can follow other users to stay updated on their activity.
Postcondition	Post is published, engagement counters are updated, and notifications are dispatched.
Alternate Flow	Designer-specific flow: designer submits a design proposal, which enters the admin approval pipeline before becoming visible to the community.

3.5.6 UC-6: Merchant Product Management

Actor	Merchant
Precondition	User has the merchant role and a registered merchant profile.
Main Flow	1. Merchant accesses the merchant dashboard. 2. Merchant creates a new product listing with all required attributes (name, description, price, images, sizes, colors, material, brand). 3. Merchant optionally adds a size measurement chart mapping sizes to body measurements. 4. Merchant manages existing listings: updates stock, adjusts prices, adds sale prices, or deactivates products. 5. Merchant views and processes incoming orders.
Postcondition	Product listings are updated and immediately reflected in the storefront and search results.

3.6 Data Model and Entity-Relationship Diagram

The system uses MongoDB as its primary data store. While MongoDB is a document-oriented database without enforced relational schemas, the application layer defines structured document schemas using Mongoose. The following entity-relationship diagram il-

illustrates the logical relationships between the core entities.

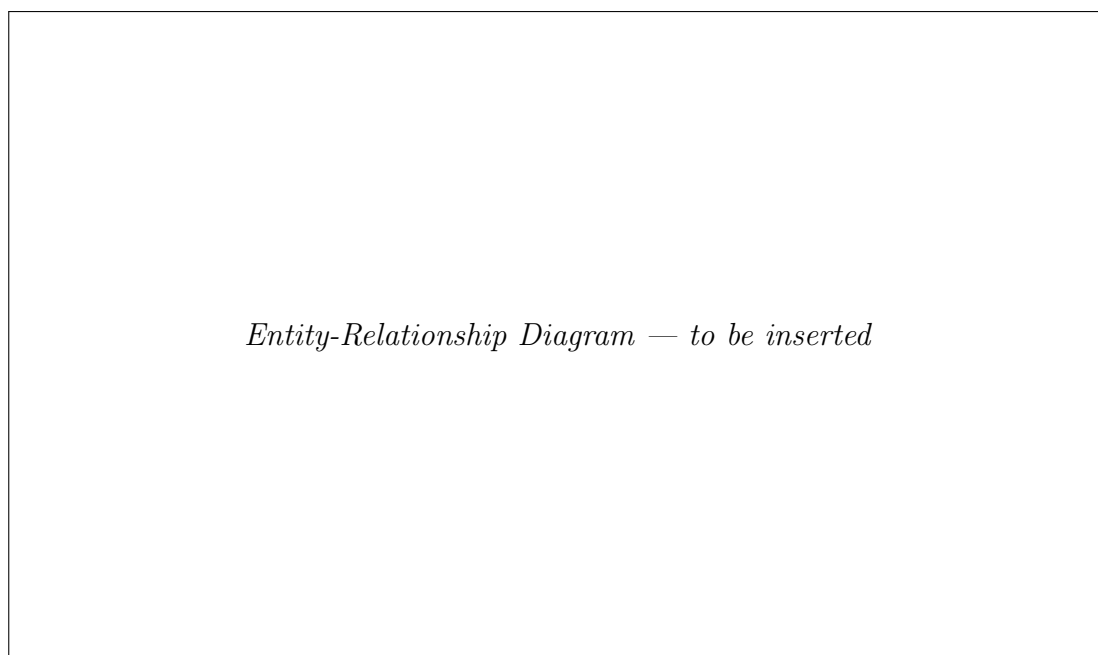


Figure 3.2: Entity-relationship diagram for Prova.

3.6.1 Core Entities

User. The central entity representing all platform users. Key attributes include name, email (with normalized duplicate), password (hashed, select-excluded), phone, address, birth date, role (user/merchant/admin/cs/designer), authentication provider (local/Google), profile images, body measurement fields (height_cm, weight_kg), a dedicated try-on image, account status flags (isActive, isSuspended), social counters (followersCount, followingCount), session tracking, and token management fields (tokenVersion, refreshToken, resetOTP).

Product. Represents a marketplace listing. Key attributes include name, description, price, sale price, stock count, merchant reference, category, garment type, image URLs, tags, available sizes, a size measurement chart (array of size-to-measurement mappings for chest, waist, hip, shoulder, inseam), colors, gender, material, brand, styles, occasions, and view count. Products are indexed on category, type, brand, price, merchant, tags, colors, and a text index on name, description, and brand for full-text search.

Order. Captures a purchase transaction. Contains a user reference, an array of order items (each with product reference, quantity, price, selected size, and selected color), total amount, status, delivery address, and payment method.

MerchantProfile. Extends a user with merchant-specific data: company name, company ID, national ID, and a list of product references.

Review. A product review containing user reference, product reference, rating (0.5–

5), comment text, likes (array of user references), and threaded replies.

TryOnResult. Records a virtual try-on session. Contains user and product references, model type (vton2d/vton360/multiangle), result image URL and public ID, video generation status and metadata, garment category, and timestamps.

BodyMeasurements. Stores measurement estimation results. Contains user reference, engine used (shapy), source image URLs, input metadata (height, weight, gender, region, fit), the full measurements object, size recommendation, confidence scores, debug information, and processing time.

3.6.2 Community Entities

Post. A community post with author reference, type (issue/review/tip/question), title, content, optional images, optional store reference, optional rating, engagement counters (likesCount, commentsCount), and active status.

Design. A designer submission with name, description, category, price, designer reference, approval status (pending/approved/in_production/published/rejected), vote counters, admin notes, and a computed virtual field for net votes.

Comment. A polymorphic comment that targets either a post or a design (via targetType and targetId). Supports threaded replies through a parentId self-reference. Tracks likesCount and repliesCount.

Vote. A polymorphic vote entity targeting posts, designs, or comments. Each vote has a type (like/upvote/downvote) and is uniquely constrained per user per target.

Follow. Represents a follower-following relationship between two users with a unique constraint preventing duplicate follows and a pre-save hook preventing self-follows.

Notification. Event-driven notifications with recipient, sender, type (like_post, comment_post, upvote_design, follow, etc.), target reference, message text, read status, and a 30-day TTL auto-expiry index.

Wishlist. A polymorphic bookmark entity allowing users to save products, posts, or designs with optional notes. Uniquely constrained per user per item type and ID.

SupportTicket. Represents a customer service interaction. Contains user reference and contact details, optional agent assignment, status (open/assigned/resolved/closed), subject, priority (low/medium/high), a threaded messages array (each message with sender ID, name, role, content, and timestamp), last message preview, and unread counters for both user and agent sides.

CustomerServiceProfile. Extends a user with the customer service role. Links the agent to a specific branch for organizational routing.

3.6.3 Key Relationships

- A **User** has zero or more **Orders**, **Reviews**, **TryOnResults**, **BodyMeasurements**, **Posts**, **Designs**, **Comments**, **Votes**, **Wishlists**, and **Notifications**.
- A **Merchant** (User with merchant role) has one **MerchantProfile** and zero or more **Products**.
- A **Product** belongs to one **Merchant** and has zero or more **Reviews** and **OrderItems**.
- An **Order** belongs to one **User** and contains one or more **OrderItems**, each referencing a **Product**.
- A **Comment** belongs to one **User** and targets one **Post** or **Design**. Comments may have a parent **Comment** for threading.
- A **Vote** belongs to one **User** and targets one **Post**, **Design**, or **Comment**.
- A **Follow** connects two **Users** (follower and following).

3.7 Class Diagram

The class diagram illustrates the major software components and their relationships across the backend and AI services.

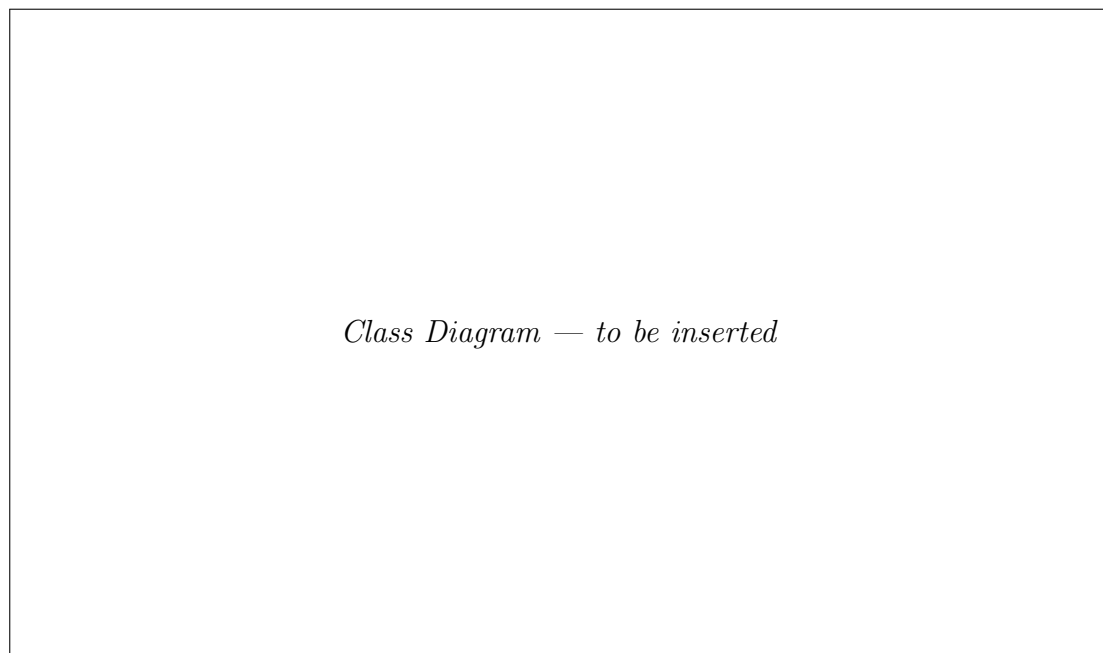


Figure 3.3: Class diagram showing major components of Prova.

The backend follows a layered module pattern. Each domain module (auth, product, order, post, etc.) contains four components: a **Routes** file that defines HTTP endpoints and attaches middleware, a **Controller** that handles request parsing and response formatting, a **Service** that implements business logic, and a **Model** that defines the Mongoose schema and database interface. Validation is handled by **Schema** files using Zod. Cross-cutting concerns—authentication, rate limiting, CORS, security headers, logging, and error handling—are implemented as Express middleware.

The try-on service follows a similar layered pattern using FastAPI. **Routers** define API endpoints, **Services** (ColabClient, MultiAngleClient, VTON360Client, SHAPY-Client, VideoPreprocessor) implement the AI pipeline logic, and **Models** define request/response schemas using Pydantic. A **Dependencies** module provides dependency injection for service instances, and a centralized **ClientState** singleton manages the life-cycle of all external service clients.

The RAG service is organized as a pipeline of composable stages: MessageClassifier, IntentExtractor, ProductRetriever, OutfitAssembler, and ResponseComposer, coordinated by a Pipeline orchestrator. Configuration, embeddings, prompts, and schemas are separated into dedicated modules.

3.8 Process and Sequence Diagrams

This section documents the key processes in the system using sequence diagrams that trace the flow of data and control across service boundaries.

3.8.1 Product Purchase Flow

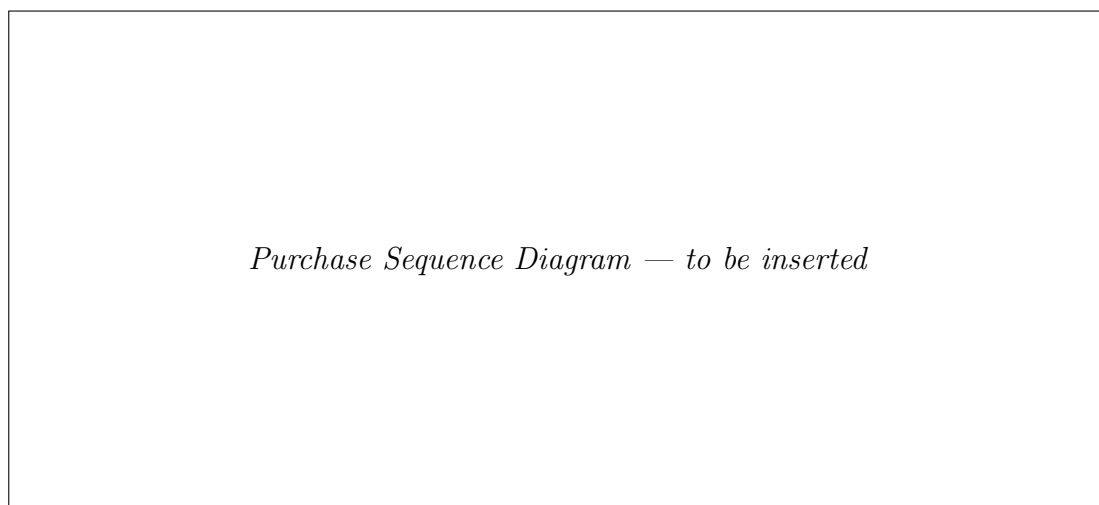


Figure 3.4: Sequence diagram for the product purchase flow.

The purchase flow proceeds as follows: (1) the shopper adds items to their cart on the frontend, (2) at checkout, the frontend sends a POST request to the backend order endpoint with items, address, and payment method, (3) the backend validates stock availability for each item, (4) the backend creates the order document and decrements stock for each product atomically, (5) the backend returns the order confirmation to the frontend, and (6) the frontend navigates to the order success page. If any item is out of stock, the backend returns a validation error before creating the order.

3.8.2 Virtual Try-On Flow

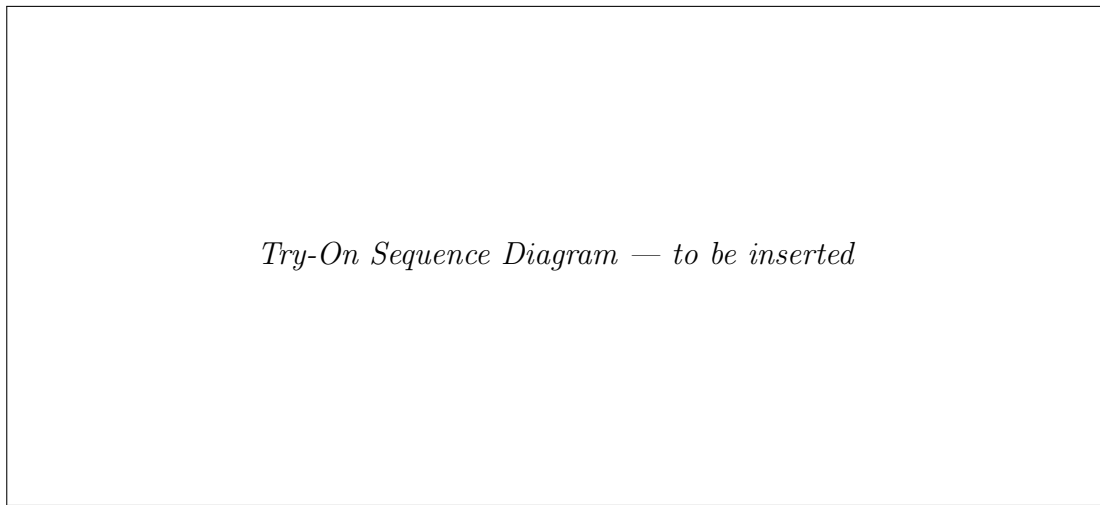


Figure 3.5: Sequence diagram for the 2D virtual try-on flow.

The 2D try-on flow involves three services: (1) the frontend uploads person and garment images to the backend, (2) the backend proxies the request to the try-on FastAPI service, (3) the try-on service validates and preprocesses both images, (4) the service sends the images to the diffusion model (OOTDiffusion) for synthesis, (5) the result image is returned to the backend, (6) the backend uploads the result to cloud storage (Cloudinary), saves a TryOnResult document, and returns the image URL to the frontend.

The measurement flow involves the following steps: (1) the frontend uploads a front-facing photograph along with height, weight, gender, and region parameters, (2) the backend forwards the request to the try-on service measurement endpoint, (3) the measurement router validates inputs and sends the image to the SHAPY service, (4) SHAPY performs 3D body mesh regression and returns raw shape parameters and circumference estimates, (5) the router applies weight-based circumference correction if the predicted mass deviates significantly from the user-provided weight, (6) derived measurements (widths, lengths) are computed using ISO 7250 anthropometric ratios, (7) the size recommendation module maps chest and waist circumferences to regional size charts with the selected fit preference, and (8) the complete result is returned with confidence scores and persisted to the database.

3.8.5 Multiview Try-On and 360 Reconstruction Flow

The multiview pipeline is the most complex workflow in the system and involves multiple stages across services: (1) the user uploads a video or set of images showing themselves from multiple angles, (2) the preprocessing service extracts frames (if video), runs person detection (YOLOv8), generates DensePose maps, and produces human parsing masks for each frame, (3) the user uploads garment front and back images, which are saved and organized, (4) the system builds a structured dataset ZIP containing the preprocessed person data and cloth images, (5) the ZIP is uploaded to the inference server, (6) the multiview try-on model generates view-consistent results across all viewpoints, (7) results are downloaded and extracted locally, and (8) the multiview outputs can be fed into the Splatfacto Gaussian splatting pipeline to produce an interactive 360-degree reconstruction.

3.9 Summary

This chapter has defined the functional and non-functional requirements for Prova, identified the stakeholders and their interactions through use cases, documented the data model through entity descriptions and relationships, outlined the software architecture through a class diagram, and traced the key system processes through sequence diagrams. These analysis artifacts establish the foundation for the system design presented in Chapter 4 and the implementation details described in Chapter 5.

Chapter 4

Proposed System Design

4.1 Overview

This chapter translates the requirements and analysis artifacts from Chapter 3 into concrete design decisions. It describes the high-level architecture, the internal structure of each service, the API design contracts, the data flow between services, the algorithmic stages within each AI pipeline, the security architecture, and the deployment topology. The design prioritizes modularity, independent scalability of AI workloads, and a clear separation between business logic and inference-heavy components.

4.2 Architecture Overview

Prova follows a **service-oriented architecture (SOA)** comprising four independently deployable services that communicate over HTTP/REST:

1. **Frontend Service** — A Next.js application that renders the user interface, manages client-side state, handles internationalization (Arabic/English with RTL support), and communicates with the backend and AI services via REST calls.
2. **Backend Service** — An Express.js application written in TypeScript that serves as the central orchestrator. It handles authentication, authorization, business logic for e-commerce and community features, proxies requests to AI services, and manages all persistent data through MongoDB.
3. **Try-On Service** — A FastAPI application that exposes specialized endpoints for virtual try-on (2D and multiview), body measurement estimation, video preprocessing, and 360-degree reconstruction. It manages connections to external inference servers and local preprocessing pipelines.

<code>product/[id]/</code>	# Product detail
<code>cart/</code>	# Shopping cart
<code>checkout/</code>	# Checkout flow
<code>orders/</code>	# Order history
<code>order-success/</code>	# Order confirmation
<code>profile/</code>	# User profile & settings
<code>virtual-tryon/</code>	# Try-on interface
<code>community/</code>	# Posts, designs, feed
<code>recommendations/</code>	# AI-powered suggestions
<code>customer-service/</code>	# Support ticket interface
<code>merchant-owner/</code>	# Merchant dashboard
<code>admin/</code>	# Admin panel
<code>login/ signup/</code>	# Authentication pages
<code>wishlist/</code>	# Saved items
<code>dashboard/</code>	# User dashboard

4.3.2 State Management and API Communication

The frontend uses React hooks and context for local state management. API communication is centralized through a typed API client module that handles:

- Attaching JWT access tokens from local storage to all authenticated requests.
- Intercepting 401 `Unauthorized` responses and checking for a refreshed token in the `X-New-Access-Token` response header (transparent token rotation).
- Providing typed request/response wrappers for each backend endpoint.

4.3.3 Internationalization (i18n)

The application supports Arabic and English through Next.js internationalized routing. Each locale has its own translation files loaded at build time. The layout component reads the current locale parameter and applies the appropriate `dir` attribute (`rtl` for Arabic, `ltr` for English) to the root HTML element. All UI text, labels, and messages reference translation keys rather than hardcoded strings.

4.4 Backend Design

4.4.1 Module Architecture

The backend follows a **layered module pattern** where each domain feature is encapsulated in its own directory under `src/modules/`. Every module contains four standard

files:

- **Routes** (`*.routes.ts`) — Defines Express router endpoints, attaches middleware (authentication, validation, rate limiting), and maps HTTP methods to controller functions.
- **Controller** (`*.controller.ts`) — Parses request parameters and body, delegates to the service layer, and formats HTTP responses.
- **Service** (`*.service.ts`) — Contains business logic, database queries, and integration calls. Services are stateless functions.
- **Model** (`*.model.ts`) — Defines the Mongoose schema, TypeScript interfaces, indexes, and virtual fields for the module's data entity.

An optional **Schema** file (`*.schema.ts`) defines Zod validation schemas that are enforced by a generic validation middleware before the request reaches the controller.

4.4.2 Middleware Pipeline

Incoming requests pass through a layered middleware pipeline before reaching route handlers:

1. **Security Headers** — Helmet middleware applies HTTP security headers (Content-Security-Policy, X-Frame-Options, etc.).
2. **CORS** — Configured to allow requests only from authorized frontend origins with credentials support.
3. **Rate Limiting** — Two tiers: a general API rate limiter and a stricter authentication-specific rate limiter for login, signup, and password reset endpoints.
4. **Body Parsing** — JSON and URL-encoded body parsing with configurable size limits.
5. **Authentication** — JWT verification with automatic token refresh. The middleware extracts the Bearer token, verifies it against the JWT secret, and attaches the decoded user object to the request. If the access token is expired but a valid refresh token cookie is present, the middleware silently issues a new access token via the `X-New-Access-Token` response header.
6. **Role Authorization** — A configurable role guard that checks whether the authenticated user's role is in the set of permitted roles for the endpoint.

7. **Validation** — Zod schema validation middleware that validates request body, query, and params against the module’s schema definitions, returning structured error responses on failure.
8. **Profanity Guard** — Content moderation middleware applied to user-generated content endpoints.

An **optional authentication middleware** variant is also provided for endpoints that enhance the experience for authenticated users but remain accessible to anonymous visitors (e.g., product browsing with personalized recommendations).

4.4.3 User Caching

To reduce database load on the authentication hot path, the backend implements an in-memory user cache. On successful JWT verification, the user document is stored in a TTL-bounded LRU cache keyed by user ID. Subsequent requests from the same user within the TTL window are served from cache, bypassing the MongoDB query. Cache entries are invalidated when the user’s token version changes (e.g., after password change or forced logout).

4.4.4 Error Handling

A global error handler middleware catches all unhandled exceptions and returns standardized JSON error responses with appropriate HTTP status codes. Domain-specific errors (e.g., `ServiceUnavailableError`, `ExternalAPIError`, `ProcessingError`) are defined as custom exception classes that carry structured metadata for debugging.

4.5 Try-On Service Design

4.5.1 Service Architecture

The Try-On Service is a **FastAPI** application organized into routers, service clients, preprocessing modules, and a centralized dependency injection container.

Routers define the API surface:

- `/api/tryon` — Single-view 2D virtual try-on using OOTDiffusion via a remote Colab inference server.
- `/api/multiangle` — Multi-angle try-on using a multimodal generative model for front, right, left, and back viewpoints.
- `/api/vton360` — Multiview pipeline with preprocessing, dataset construction, remote inference, and result retrieval.

- `/api/measurements` — Body measurement estimation using the SHAPY client with size recommendation mapping.
- `/api/video360` — 360-degree video generation from multiview try-on outputs.
- `/api/health` — Aggregated health check across all service clients.

Service Clients encapsulate communication with external inference services. Each client follows a consistent pattern:

- Configurable base URL, timeouts, and connection pool limits via environment variables.
- Automatic retry with exponential backoff for transient failures (HTTP 429, 5xx).
- Health check methods for proactive monitoring.
- Structured exception hierarchy (`ServiceUnavailableError`, `ExternalAPIError`).

A **ClientState** singleton manages the lifecycle of all service clients. It is initialized once at application startup and provides dependency injection to routers via FastAPI’s dependency system.

4.5.2 Multi-Angle Try-On Pipeline Design

The multi-angle try-on client implements a two-stage pipeline for producing realistic virtual try-on results across multiple viewpoints:

Stage 1 — Garment Preparation. Before try-on synthesis, the system determines whether the garment reference image contains a model wearing the garment or is a flat product shot. The decision is made through a three-step heuristic:

1. Check for transparent background (alpha channel analysis) — transparent images are classified as flat product shots.
2. Detect skin-tone pixels using RGB-space rules and check for portrait aspect ratio — images with significant skin content in portrait orientation are classified as model-worn.
3. If the source mode is ambiguous (`auto`), the heuristic result is used; explicit modes (`flat_product`, `model_worn`) override the heuristic.

For model-worn garment references, an extraction step uses the multimodal model to isolate the garment from the person, producing a clean product image before try-on synthesis.

Stage 2 — Try-On Synthesis. The prepared garment image and the person image are submitted to the generative model with a composed prompt. The prompt is dynamically constructed from a base template augmented with control parameters:

- **Garment type** (upper/lower/dress/shoes) — instructs the model on which body region to modify.
- **Shirt placement** (tucked/untucked) — controls hem behavior.
- **View hint** (front/right/left/back) — preserves the camera orientation of the person image.

For multi-angle try-on, the system processes multiple viewpoints concurrently using an asyncio semaphore (concurrency limit of 2) to balance throughput against API rate limits. Garment preparation is performed once and shared across all views.

4.5.3 Multiview (VTON360) Pipeline Design

The multiview pipeline is designed for generating view-consistent try-on results across all angles. It operates in five stages:

1. **Preprocessing** — Input video or images are processed through YOLOv8 for person detection, Detectron2 for DensePose map generation, and a human parsing network for segmentation masks. Results are organized into a structured directory hierarchy.
2. **Cloth Preparation** — Garment front and back images are saved, resized, and organized into the dataset structure expected by the inference model.
3. **Dataset Construction** — Person preprocessing outputs and cloth images are assembled into a ZIP archive following the model’s expected directory format, ready for upload.
4. **Remote Inference** — The ZIP is uploaded to the inference server. The service polls for job completion with configurable timeouts.
5. **Result Retrieval** — Completed results are downloaded, extracted, and made available through the API. The multiview outputs can optionally be fed into the Splatfacto 3D Gaussian splatting pipeline for 360-degree interactive reconstruction.

4.5.4 Measurement Pipeline Design

The measurement pipeline transforms a single front-facing photograph into body measurements and size recommendations through the following stages:

1. **Input Validation** — The router validates image format, file size, height (in cm), weight (in kg), and gender parameters.

2. **SHAPY Inference** — The image and metadata are sent to the SHAPY service, which performs SMPL-X body mesh regression and returns raw shape parameters and circumference estimates.
3. **Weight-Based Correction** — If the predicted body mass deviates significantly from the user-provided weight, the router applies a proportional correction factor to circumference measurements. This compensates for the inherent ambiguity of estimating depth information from a single 2D image.
4. **Derived Measurements** — Additional body dimensions (shoulder width, arm length, trouser length, thigh circumference) are computed from the corrected circumferences using ISO 7250 anthropometric ratios.
5. **Size Recommendation** — Chest and waist circumferences are mapped to regional size charts (EU, US, UK, IT, FR, DE) with configurable fit preferences (regular, slim, oversized). The fit preference applies a scaling factor: slim reduces the target circumference, oversized increases it.
6. **Confidence Scoring** — Each measurement receives a confidence score based on the quality of the input image, the magnitude of corrections applied, and the consistency between predicted and reported anthropometric data.

4.6 Chat Service (RAG) Design

4.6.1 Pipeline Architecture

The RAG service implements a five-stage pipeline pattern where each stage is a composable module with a defined input/output contract:

1. **Message Classifier** — Categorizes incoming messages into one of five types: *product_search*, *greeting*, *follow-up*, *non-fashion*, or *clarification*. Non-search intents receive immediate responses without triggering the retrieval pipeline, reducing unnecessary computation.
2. **Intent Extractor** — For product search messages, extracts a structured query object containing: garment type, colors, budget range, sizes, brands, occasion, gender, style preferences, and any free-text description. The extraction uses a large language model with a constrained output schema.
3. **Product Retriever** — Executes a two-phase hybrid search:

- *Phase 1 (Metadata Filtering)*: Constructs a MongoDB query from the extracted intent attributes, applying filters on category, type, gender, brand, color, size, and price range.
 - *Phase 2 (Semantic Ranking)*: Computes cosine similarity between the query embedding (generated by Sentence-BERT) and pre-computed product embeddings. Products below a configurable similarity threshold are discarded.
4. **Outfit Assembler** — Groups retrieved products into coherent outfit combinations. The assembler attempts to match complementary garment types (e.g., a top with a bottom) and applies diversity constraints to avoid recommending multiple similar items.
 5. **Response Composer** — Generates a bilingual natural language response grounded in the retrieved products. The composer receives the outfit selections, the user’s original query, conversation history, and user profile data (if available). It produces explanations for why each item was recommended, styling tips based on body type, and explicit disclaimers when no suitable products are found.

4.6.2 Conversation History

The service maintains per-user conversation history in an in-memory LRU (Least Recently Used) store. The store is keyed by `user_id` (primary) or `session_id` (fallback), with configurable limits: up to 5,000 distinct keys and 20 turns (user + assistant messages) per key. Older entries are evicted when limits are reached. This enables multi-turn conversations where the assistant can reference previous queries without requiring external storage.

4.6.3 Knowledge Base Integration

A fashion-specific knowledge base provides profile-aware styling guidance. The knowledge base contains rules indexed by body type, height range, and gender that map to recommended silhouettes, colors, and fit preferences. When the user’s profile includes height, weight, and gender, the retriever consults the knowledge base to augment recommendations with personalized styling advice.

4.6.4 Embedding Strategy

Product embeddings are pre-computed using Sentence-BERT (`all-MiniLM-L6-v2`) over a concatenation of product name, description, brand, tags, styles, and occasion fields. Embeddings are stored alongside product documents in MongoDB. A dedicated admin

endpoint (POST `/precompute-embeddings`) triggers bulk re-computation when the product catalog is updated.

4.7 API Design

The system exposes RESTful APIs following consistent design conventions:

4.7.1 Backend API Conventions

- **URL Structure:** Resources follow a noun-based hierarchy (e.g., `/api/products`, `/api/orders/:id`, `/api/community/posts`).
- **Authentication:** Protected endpoints require a **Bearer** token in the **Authorization** header. Public endpoints use the optional auth middleware.
- **Response Format:** All responses follow a consistent JSON structure with **success** (boolean), **msg** (human-readable message), and **data** (payload) fields.
- **Error Format:** Errors include **success: false**, a machine-readable **code** (e.g., `TOKEN_EXPIRED`), and a descriptive **msg**.
- **Pagination:** List endpoints support **page** and **limit** query parameters with default values.
- **Validation:** Request bodies are validated against Zod schemas before reaching controllers.

4.7.2 Try-On Service API

The Try-On Service accepts multipart form data for image uploads. Key endpoints include:

Endpoint	Method	Description
<code>/api/tryon/try-on</code>	POST	Submit person and garment images for 2D try-on using OOTDiffusion.
<code>/api/multiangle/try-on</code>	POST	Single-view try-on with garment type, source mode, and placement controls.
<code>/api/multiangle/multi</code>	POST	Multi-angle try-on across front, right, left, and back viewpoints.
<code>/api/measurements/measure</code>	POST	Body measurement estimation from a front-facing photograph.

Endpoint	Method	Description
/api/vton360/preprocess	POST	Preprocess person video/images for the multiview pipeline.
/api/vton360/upload	POST	Upload garment front and back images.
/api/vton360/build	POST	Assemble preprocessed data into inference-ready format.
/api/vton360/infer	POST	Submit dataset for multiview inference.
/api/vton360/results	GET	Retrieve completed inference results.
/api/health	GET	Aggregated health status across all service clients.

4.7.3 Chat Service API

Endpoint	Method	Description
/chat	POST	Send a natural language message through the RAG pipeline. Accepts message , userId , sessionId , and lang .
/health	GET	Service health check with uptime reporting.
/precompute-embeddings	POST	Admin utility to re-compute all product embeddings.

4.8 Security and Privacy Design

4.8.1 Authentication Flow

The system implements a dual-token authentication strategy:

- **Access Token** — A short-lived JWT (configurable TTL, typically 15 minutes) containing **userId** and **tokenVersion**. Sent in the **Authorization: Bearer** header.
- **Refresh Token** — A longer-lived JWT stored as an HTTP-only cookie. Used to silently rotate access tokens when they expire.
- **Token Versioning** — Each user has a **tokenVersion** counter. Changing the password or performing a forced logout increments this version, immediately invalidating all existing tokens.

4.8.2 Password Security

Passwords are hashed using **bcrypt** with a configurable salt rounds parameter before storage. Raw passwords are never persisted or logged. The password field is excluded from all database queries by default using Mongoose's `select: false` option.

4.8.3 Rate Limiting

Two tiers of rate limiting protect the system:

- **General API rate limiter** — Applied to all endpoints with a generous window to prevent abuse while allowing normal usage patterns.
- **Authentication rate limiter** — Stricter limits on login, signup, and password reset endpoints to mitigate brute-force attacks. A separate **password reset rate limiter** is applied to the forgot-password, verify-code, and reset-password flow.

4.8.4 Input Validation and Sanitization

All user inputs are validated at two levels:

- **Schema validation** (Zod) — Ensures type safety, required fields, string length limits, and format constraints (e.g., email format, phone number patterns).
- **Content moderation** — A profanity guard middleware scans user-generated text content before it reaches the database.

4.8.5 Data Privacy

Uploaded images for try-on and body measurements are processed in-memory where possible and stored in Cloudinary with access-controlled URLs. Try-on results are associated with authenticated user accounts, ensuring that results are only accessible to the user who generated them. The body measurement pipeline does not retain raw images beyond the inference session unless explicitly saved by the user.

4.9 Deployment Architecture

The system is designed for deployment across multiple environments:

- **Frontend** — Deployed as a containerized Next.js application behind a reverse proxy (Nginx or Vercel). Static assets are served from a CDN.
- **Backend** — Runs as a Node.js process with PM2 for process management and automatic restarts. Connects to a managed MongoDB Atlas cluster.

- **Try-On Service** — Runs as a Uvicorn ASGI server. GPU-intensive inference (OOT-Diffusion, VTON360) is offloaded to remote Colab or dedicated GPU servers accessed via HTTP. The local service handles preprocessing, orchestration, and result delivery.
- **Chat Service** — Runs as a Uvicorn ASGI server. The Sentence-BERT embedding model runs locally on CPU. LLM inference is delegated to an external API endpoint.
- **Database** — MongoDB Atlas provides the primary data store with automated backups, replica sets for high availability, and Atlas Search for text indexing.
- **Media Storage** — Cloudinary serves as the cloud media storage provider for product images, profile photos, try-on results, and community post images.

4.10 Summary

This chapter has presented the system design for Prova, covering the service-oriented architecture, the internal design of each service (frontend routing and i18n, backend module pattern and middleware pipeline, try-on multi-stage pipelines, and RAG pipeline stages), the API contracts, the security architecture, and the deployment topology. These design decisions directly implement the requirements defined in Chapter 3 and provide the blueprint for the implementation described in Chapter 5.

Chapter 5

Implementation

5.1 Overview

This chapter describes how Prova is implemented across the frontend, backend, and AI services. It also documents the models, datasets, and results that validate the marketplace, RAG retrieval, multiview 2D try-on, measurement estimation, and 360 view features.

5.2 Backend Implementation

The backend is written in TypeScript using Express. It provides REST endpoints for authentication, product and order management, community content, and proxying AI requests. Validation is performed using schema-based checks, and MongoDB is used for persistence.

5.3 Frontend Implementation

The frontend uses Next.js with the App Router. It integrates localization, UI components, and interactive workflows for marketplace browsing, community engagement, designer pages, and 2D try-on.

5.4 Try-On Service Implementation

The FastAPI try-on service exposes endpoints for the custom multiview 2D try-on pipeline. The implementation follows the same high-level stages described in the VTON360 research paper, including preprocessing, conditioning, synthesis, and post-processing. A splatfacto-based module uses multiview outputs to reconstruct a full 360 view.

5.5 Body Measurement Implementation

The measurement module estimates key body measurements from images and maps them to size recommendations. This section will detail the measurement pipeline, inputs, outputs, and the sizing rules used by the system.

5.6 RAG System Implementation

The retrieval system ingests product data into a vector store and performs semantic search to surface relevant items. It is integrated into the marketplace experience to support discovery and recommendation flows.

5.7 Models and Equations

This section will detail the models used in the system and the key equations involved in try-on conditioning, measurement estimation, 360 reconstruction, and retrieval scoring.

5.8 Datasets

This section will document datasets used for evaluation, including sources, sizes, preprocessing steps, and splits.

5.9 Results

This section will present quantitative metrics, qualitative results, and sample outputs for each subsystem.

Chapter 6

Testing

6.1 Testing Strategy

Testing focuses on functional correctness, integration stability, and performance under realistic workloads.

6.2 Unit Testing

Unit tests validate core utilities, data preprocessing, and request validation logic.

6.3 Integration Testing

Integration tests verify end-to-end flows across frontend, backend, and AI services, including error handling.

6.4 Performance Testing

Performance measurements capture response times for try-on pipelines, chatbot latency, and throughput under concurrent requests.

6.5 User Testing

User feedback sessions evaluate usability, perceived realism of try-on results, and satisfaction with chatbot responses.

6.6 Summary

The testing results demonstrate the stability of the system and highlight areas for future optimization.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project delivers an integrated platform that combines virtual try-on, measurement estimation, and conversational assistance for fashion e-commerce. The modular architecture enables flexible deployment and sets a foundation for future improvements.

7.2 Future Work

Future enhancements include higher-fidelity try-on models, improved multi-view consistency, tighter latency guarantees, and richer personalization using user feedback and analytics.

Bibliography

- [1] X. Han, Z. Wu, Z. Wu, R. Yu, and L. S. Davis, “Viton: An image-based virtual try-on network,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.
- [2] B. Wang, H. Zheng, X. Liang, Y. Chen, L. Lin, and M. Yang, “Toward characteristic-preserving image-based virtual try-on network,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.
- [3] S. Lee, G. Gu, S. Park, S. Choi, and J. Choo, “High-resolution virtual try-on with misalignment and occlusion-handled conditions,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.
- [4] J. Kim, G. Gu, M. Park, S. Park, and J. Choo, “Stableviton: Learning semantic correspondence with latent diffusion model for virtual try-on,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [5] Y. Choi, S. Kwak, K. Lee, H. Choi, and J. Shin, “Improving diffusion models for authentic virtual try-on in the wild,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [6] Y. Xu, T. Gu, W. Chen, and C. Chen, “Ootdiffusion: Outfitting fusion based latent diffusion for controllable virtual try-on,” in *arXiv preprint arXiv:2403.01779*, 2024.
- [7] Z. He, Y. Ning, Y. Qin, *et al.*, *Vton 360: High-fidelity virtual try-on from any viewing direction*, 2025. arXiv: 2503.12165 [cs.CV].
- [8] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, 2020.
- [9] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [10] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020.

- [11] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.
- [12] M. Tancik, E. Weber, E. Ng, *et al.*, “Nerfstudio: A modular framework for neural radiance field development,” in *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.
- [13] R. A. Güler, N. Neverova, and I. Kokkinos, “Densepose: Dense human pose estimation in the wild,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018.
- [14] M. Loper, N. Mahmood, J. Romero, G. Pons-Moll, and M. J. Black, “Smpl: A skinned multi-person linear model,” *ACM Transactions on Graphics*, vol. 34, no. 6, 2015.
- [15] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, “Detectron2,” <https://github.com/facebookresearch/detectron2>, 2019.
- [16] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2016.
- [17] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolov8*, <https://github.com/ultralytics/ultralytics>, 2023.
- [18] V. Choutas, L. Müller, C.-H. P. Huang, S. Tang, D. Tzionas, and M. J. Black, “Accurate 3d body shape regression using metric and semantic attributes,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [19] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [20] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019.